

GitOps at Scale:

Domain-Driven Argo CD Hub-and-Spoke Architecture on Red Hat OpenShift at Bradesco

Authors: Your Name, Co-author Name

Organization: Your Company / Red Hat Partner

Date: September 29, 2026

Version: 1.0

Abstract

This article describes the migration from a per-environment Argo CD topology — one instance each for development, staging, and production — to a domain-driven hub-and-spoke architecture on Red Hat OpenShift. The original model concentrated all business-domain application workloads into a single Argo CD controller per environment, creating a sync bottleneck and cascading failures in GitOps workflows. The solution introduces a central *hub* Argo CD instance that provisions domain-scoped *spoke* Argo CD instances via **ApplicationSet** path generators, backed by a standardized Helm chart with sub-charts and a path-structured application registry. The result is fault-isolated, independently scalable GitOps domains with a dramatically reduced blast radius for controller failures.

Contents

1	Introduction	2
2	Problem Statement	2
2.1	The per-environment topology and its limits	2
2.2	Observed failure modes	2
2.3	Requirements for the new model	3
3	Solution Architecture	3
3.1	Domain-driven hub-and-spoke overview	3
3.2	Component mapping	4
3.3	Repository structure	4
3.4	Design decisions	4
4	Implementation	4
4.1	Prerequisites	5
4.2	Installing the hub Argo CD	5
4.3	ApplicationSet — provisioning spoke instances	5
4.4	Helm sub-chart: argocd-spoke	6
4.5	Spoke ApplicationSet — deploying domain applications	6
5	Challenges and Mitigations	6
5.1	Challenge 1: Migration without downtime	6
5.2	Challenge 2: Hub ApplicationSet reconciliation during bulk domain import	6
5.3	Challenge 3: Helm sub-chart values inheritance	7
5.4	Challenge 4: Spoke Argo CD RBAC bootstrapping	7
6	Results and Outcomes	7
7	Lessons Learned	7
8	Conclusion	7
	Acknowledgements	7
	References	8

1. Introduction

Modern software organizations operating on Red Hat OpenShift typically evolve through predictable stages of GitOps adoption. The first stage is “environment-centric” GitOps: one Argo CD instance manages all applications headed for a given environment — development, staging, and production. This model is easy to reason about initially, but it collapses under the weight of a growing application portfolio because a single controller becomes the shared failure domain for every business domain in the organization.

Our organization reached that inflection point when the production Argo CD instance began exhibiting systematic sync failures under the combined load of all business-domain workloads. Webhook-triggered workflows, image-updater automations, and manual syncs contended for the same reconciliation queue, producing cascading timeouts that were difficult to attribute to any single team or application.

This article documents our migration to a *domain-driven hub-and-spoke* topology in which the hub Argo CD instance is responsible exclusively for provisioning and maintaining spoke Argo CD instances, each scoped to a single business domain. Applications are deployed by the spoke Argo CD instance that owns the domain, not by a shared environment-level controller.

2. Problem Statement

2.1. The per-environment topology and its limits

The original deployment model allocated one Argo CD instance per environment:

- `argocd-dev` — managed all development-environment applications across every business domain.
- `argocd-staging` — identical scope for the staging environment.
- `argocd-prod` — identical scope for the production environment.

Each instance was configured identically and independently maintained. As the number of applications and teams grew, several structural weaknesses emerged.

2.2. Observed failure modes

Reconciliation bottleneck.

All domains shared a single `argocd-application-controller`. During peak hours — deploy trains, sprint releases, overnight batch promotions — the reconciliation queue depth spiked, delaying sync completion by up to 40 minutes.

Cascading sync failures.

A misbehaving `Application` from one domain (e.g. an oversized `ConfigMap` triggering a server-side diff timeout) blocked reconciliation for unrelated domains, violating the principle of fault isolation.

Workflow integration breakage.

CI/CD pipelines that called `argocd app sync` with `-timeout` observed spurious failures because the controller was busy with other tenants. This eroded confidence in GitOps as a reliable delivery primitive.

2.3. Requirements for the new model

Table 1 lists the requirements that drove the architectural decision.

Table 1: Requirements for the domain-driven GitOps architecture

ID	Category	Requirement
R-01	Isolation	A failure in one domain's Argo CD must not affect other domains
R-02	Scalability	Adding a new domain must not require changes to existing instances
R-03	Standardization	All spoke instances must share a common base configuration
R-04	Automation	Spoke provisioning must be fully GitOps-driven (no manual steps)
R-05	Observability	Sync status must be visible per domain and per environment
R-06	DR	Hub failure must not affect in-flight spoke reconciliations

3. Solution Architecture

3.1. Domain-driven hub-and-spoke overview

The new topology replaces the three shared environment instances with a two-tier hierarchy: a single *hub* Argo CD instance that owns the lifecycle of *spoke* Argo CD instances, each scoped to a business domain.

- **Hub Argo CD** — lives on the hub OpenShift cluster. Its sole responsibility is to deploy and reconcile spoke Argo CD instances (and their supporting resources) from the Helm chart repository. It does *not* deploy business applications directly.
- **Spoke Argo CD (per domain)** — one instance per business domain, deployed on the appropriate cluster. Each spoke instance manages only the applications belonging to its domain, reading from the application repository under the path that corresponds to the domain.

Figure 1 shows the logical architecture.

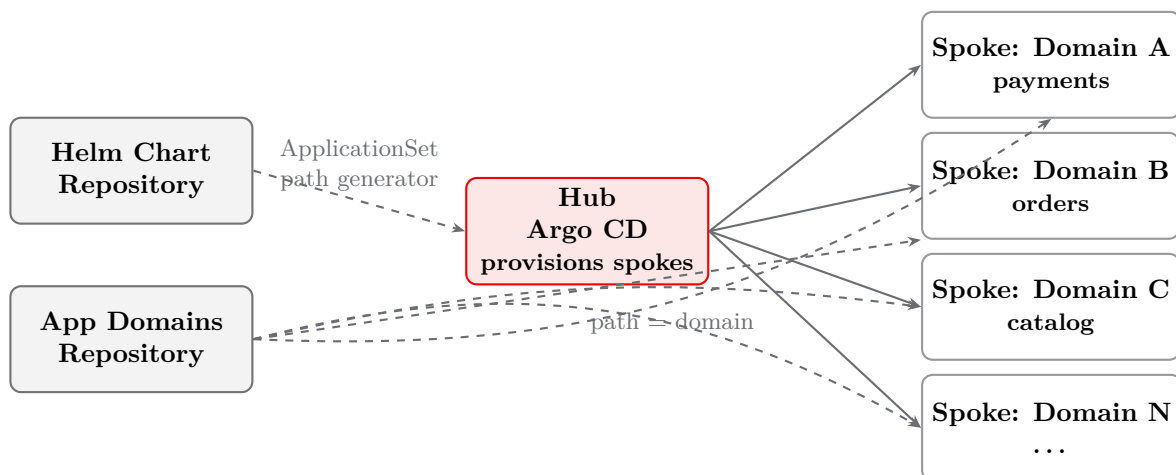


Figure 1: Domain-driven hub-and-spoke: the hub Argo CD deploys spoke instances from the Helm chart repo; each spoke reads its applications from the App Domains repo under the path matching its domain.

3.2. Component mapping

Table 2: Component roles in the domain-driven architecture

Component	Layer	Role
Hub Argo CD	Control plane	Provisions and reconciles all spoke Argo CD instances; owns no business apps
ApplicationSet (hub)	Automation	Uses a path generator over the Helm chart repo to create one spoke Application per discovered domain directory
Helm Chart Repository	Standardization	Umbrella chart with sub-charts that define the canonical Argo CD spoke and hub configuration; versioned and shared across all domains
App Domains Repository	Application inventory	Path-structured list of application; each top-level directory is a domain; each spoke Argo CD targets its own directory
Spoke Argo CD (per domain)	Delivery	Deploys and reconciles the applications declared in its domain path; isolated controller, queue, and RBAC
ApplicationSet (spoke)	Automation	Uses a path generator over the App Domains to create one Application per service declared under the domain

3.3. Repository structure

The architecture relies on three Git repositories with clearly separated concerns.

3.4. Design decisions

Domain as the isolation boundary, not environment:

Environments (dev, staging, prod) are expressed as Helm values or Kustomize overlays within each domain’s spoke, not as separate Argo CD instances. This preserves operational simplicity while achieving fault isolation at the business domain level — the boundary that actually matters for incident response.

Hub Argo CD manages only spokes:

Keeping business applications out of the hub’s reconciliation scope prevents hub saturation and makes the hub’s health directly correlated with spoke availability. If the hub is degraded, running spokes continue to reconcile their applications uninterrupted (R-06).

Path generator over cluster/list generators:

The `git` path generator discovers new domains automatically when a new directory is added to the Helm chart repo’s `domains/` tree or to the App Domains. This satisfies R-02 and R-04 without any changes to existing `ApplicationSet` manifests.

Sub-charts for standardization:

Encoding Argo CD configuration in a shared Helm sub-chart (rather than raw YAML per domain) ensures that a single `argocd-spoke` chart version pins the Argo CD release for all spokes simultaneously, making upgrades a one-commit operation (R-03).

4. Implementation

4.1. Prerequisites

- Red Hat OpenShift on the hub cluster (version *X.Y*)
- OpenShift GitOps Operator installed on the hub (provisions the hub Argo CD)
- Hub cluster has network reachability to all spoke cluster API servers
- The three Git repositories are accessible from the hub cluster
- Helm *X.Y* available in the CI environment for chart linting and testing

4.2. Installing the hub Argo CD

4.3. ApplicationSet — provisioning spoke instances

The hub Argo CD uses a `git` path generator that scans the `domains/` directory of the Helm chart repository. Each discovered sub-directory becomes one `Application` that deploys the `argocd-spoke` sub-chart with that domain's values override.

```

1 apiVersion: argoproj.io/v1alpha1
2 kind: ApplicationSet
3 metadata:
4   name: spoke-provisioner
5   namespace: openshift-gitops
6 spec:
7   goTemplate: true
8   goTemplateOptions: ["missingkey=error"]
9   generators:
10    - git:
11      repoURL: https://github.com/<org>/argo-helm-charts.git
12      revision: HEAD
13      directories:
14        - path: domains/*
15   template:
16     metadata:
17       name: "spoke-{{ .path.basename }}"
18     spec:
19       project: default
20       source:
21         repoURL: https://github.com/<org>/argo-helm-charts.git
22         targetRevision: HEAD
23         path: "."
24         helm:
25           releaseName: "argocd-{{ .path.basename }}"
26           valueFiles:
27             - values.yaml
28             - "domains/{{ .path.basename }}/values.yaml"
29       destination:
30         server: https://kubernetes.default.svc
31         namespace: "argocd-{{ .path.basename }}"
32       syncPolicy:
33         automated:
34           prune: true
35           selfHeal: true
36       syncOptions:
37         - CreateNamespace=true
38         - ServerSideApply=true

```

Listing 1: Hub ApplicationSet: path generator over the Helm chart repo

How new domains are onboarded. Adding a new business domain requires only two steps: (1) create `domains/<new-domain>/values.yaml` in the Helm chart repo, and (2) create the corresponding top-level directory in the App Domains repo. The hub `ApplicationSet` detects the new path within `requeueAfterSeconds` and provisions the spoke automatically. No changes to existing manifests are needed.

4.4. Helm sub-chart: argocd-spoke

4.5. Spoke ApplicationSet — deploying domain applications

Each spoke Argo CD instance contains a single `ApplicationSet` generated by the `argocd-spoke` sub-chart. It uses a `git` path generator over the App Domains repository, scoped to the domain's sub-tree.

```

1 apiVersion: argoproj.io/v1alpha1
2 kind: ApplicationSet
3 metadata:
4   name: {{ .Values.domain.name }}-apps
5   namespace: {{ .Release.Namespace }}
6 spec:
7   goTemplate: true
8   generators:
9     - git:
10       repoURL: https://github.com/<org>/argo-apps.git
11       revision: HEAD
12       directories:
13         - path: "{{ .Values.domain.name }}/*"
14   template:
15     metadata:
16       name: "{{ .path.basename }}"
17     spec:
18       project: {{ .Values.domain.name }}
19       source:
20         repoURL: "{{ .path.segments 1 | printf \"https://\" }}
21           ↪ github.com/<org>/%s\" }}"
22         targetRevision: HEAD
23         path: "."
24       destination:
25         server: https://kubernetes.default.svc
26         namespace: "{{ .path.basename }}"
27       syncPolicy:
28         automated:
29           prune: true
30           selfHeal: true
31       syncOptions:
32         - CreateNamespace=true
33         - ServerSideApply=true

```

Listing 2: Spoke `ApplicationSet` template inside `argocd-spoke` sub-chart

5. Challenges and Mitigations

5.1. Challenge 1: Migration without downtime

5.2. Challenge 2: Hub `ApplicationSet` reconciliation during bulk domain import

5.3. Challenge 3: Helm sub-chart values inheritance

5.4. Challenge 4: Spoke Argo CD RBAC bootstrapping

6. Results and Outcomes

7. Lessons Learned

- L-1. Isolate by failure domain, not by environment.** The environment axis (dev/staging/prod) is a deployment concern handled by values overlays. The fault-isolation axis is the business domain — the boundary along which teams own their SLAs. Aligning Argo CD instances with business domains directly maps controller scope to incident ownership.
- L-2. The hub must not host business workloads.** Any business `Application` on the hub couples its failure surface to the spoke provisioning plane. Keep the hub's `Application` list exclusively populated by `spoke-provisioner`.
- L-3. Helm sub-charts are the right abstraction for multi-instance standardization.** A shared sub-chart version is a stronger guarantee of configuration parity than documented conventions or copy-pasted YAML. Invest in `values.schema.json` early to surface mistakes at lint time rather than at runtime.
- L-4. Path generators scale better than cluster/list generators for homogeneous workloads.** Adding a domain is a file-system operation (create a directory). There is no registry, no `ConfigMap`, and no generator configuration to update. The convention-over-configuration approach pays compound interest as the number of domains grows.
- L-5. Migrate one domain at a time, with sync disabled.** The three-phase migration (disable, mirror, cut over) is slower than a bulk import but eliminates the risk of a double-sync that temporarily brings down a production service. Automate the phase sequence in a script and require a PR approval gate between phases.

8. Conclusion

The migration from per-environment to domain-driven hub-and-spoke Argo CD resolved the two most critical operational problems: reconciliation saturation and cross-domain blast radius. By assigning each business domain its own Argo CD controller — provisioned automatically by the hub via an `ApplicationSet` path generator and standardized by a shared Helm sub-chart — the platform team eliminated both the shared failure domain and the manual onboarding toil that had accumulated under the old model.

For teams considering the same journey, the most important realization is that the natural unit of GitOps isolation is not the deployment environment but the organizational boundary — the team or domain that owns incident response. Building Argo CD topology to match that boundary makes the system both more resilient and more comprehensible.

Acknowledgements

The authors thank the Red Hat OpenShift and GitOps engineering teams for their documentation and community support throughout this implementation.

References

References

- [1] Argo CD. *Argo CD — Declarative GitOps CD for Kubernetes*. <https://argo-cd.readthedocs.io>
- [2] Argo CD. *ApplicationSet Controller — Git Generator*. <https://argocd-applicationset.readthedocs.io/en/stable/Generators-Git/>
- [3] Red Hat. *OpenShift GitOps Operator Documentation*. <https://docs.openshift.com/gitops>
- [4] Helm. *Chart Dependencies (Sub-charts)*. https://helm.sh/docs/chart_template_guide/subcharts_and_globals/
- [5] OpenGitOps. *GitOps Principles v1.0*. Cloud Native Computing Foundation, 2021.
- [6] Argo CD. *Best Practices*. https://argo-cd.readthedocs.io/en/stable/user-guide/best_practices/